SLO-03: Composition and Error Budgets

> **Series:** SLO — Service Level Objectives | **Notebook:** 3 of 6 | **Created:** June 2026 | **Last Updated:** 08/31/2026

Overview

A single SLI rarely captures a service's health, and a raw SLI percentage is a weak alert signal. This notebook covers the two ideas that make SLOs operationally useful: the **error budget** (and its **burn rate**), and **composition** — combining SLIs into composite and weighted SLOs. The burn-rate query here is validated against a live tenant; the weighted-global calculation is worked end to end with an honest note on what Dynatrace does and does not compute for you.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with Grail and the SLO app
Permissions	`storage:metrics:read`
Prior reading	SLO-01 (fundamentals), SLO-02 (SLIs)

1. Error Budget Math

The error budget is the inverse of the target:

```
error budget = 100% − SLO target
```

Target	Error budget	Allowed downtime per 30 days
99%	1%	~7.2 hours
99.5%	0.5%	~3.6 hours
99.9%	0.1%	~43 minutes
99.95%	0.05%	~22 minutes

The budget reframes the conversation. Instead of "any error is a failure," the team has a quantity to spend: as long as budget remains, ship features; when it runs low, shift to hardening. This is the mechanism that connects reliability to release decisions.

2. Burn Rate

Burn rate is how fast you are consuming the budget relative to the sustainable pace. A burn rate of `1` spends the whole budget exactly over the window; `14` empties a 30-day budget in roughly two days.

```
burn rate = observed bad ratio ÷ error budget
```

The following computes the current burn rate against a 99.5% target (0.5% budget):

```
// Error-budget burn rate against a 99.5% target
// Re-validated on a live tenant 08/28/2026: observed_bad 0.045, burn_rate
9.02x
// Live figures move between runs (06/2026: ~0.041 / ~8.3x) — what should hold
// is the relationship: burn_rate = observed_bad / (1 - target)
timeseries {
  total = sum(dt.service.request.count),
  failures = sum(dt.service.request.failure_count)
}, from:-24h, interval:1h
| fieldsAdd bad_ratio = failures[] / total[]
| fieldsAdd observed_bad = arrayAvg(bad_ratio)
| fieldsAdd error_budget = 1.0 - 0.995
| fieldsAdd burn_rate = observed_bad / error_budget
| fields observed_bad, error_budget, burn_rate
```

A burn rate above `1` sustained over the window means you are on track to breach. SLO-04 turns this into multiwindow fast-burn / slow-burn alerts — the burn rate is the quantity those alerts threshold on.

3. Composite SLOs — Multiple SLIs

A service can be "available" yet unusably slow. One indicator is rarely enough. Two ways to combine:

- **Multiple separate SLOs per service** — an availability SLO *and* a latency SLO, each with its own budget. Simplest; each fails independently and routes independently. Recommended default.
- **Composite SLO** — combine SLIs into one health number. Useful for an executive roll-up where "is this service healthy?" needs a single answer. The cost is that a composite hides *which* dimension failed, so keep the constituent SLOs too.

Reach for separate SLOs for operational alerting (you want to know *what* broke); reach for a composite only when you specifically need a single combined health figure.

4. Weighted Global SLO

To express the reliability of a whole product made of several services of differing importance, weight each service's SLO and combine. Worked example for three services:

Service	Uptime target	Weight
Mobile service	99%	1.5
Web service	99.5%	1
Telephony service	98%	0.75

Multiply each target by its weight, sum, then divide by the sum of the weights:

$$\frac{(99 \times 1.5) + (99.5 \times 1) + (98 \times 0.75)}{1.5 + 1 + 0.75} = \frac{148.5 + 99.5 + 73.5}{3.25} = \frac{321.5}{3.25} = 98.92\%$$

The global SLO is **98.92%** — the overall measure weighted by each service's importance.

> **Honest caveat:** Dynatrace does **not** natively compute a weighted-global SLO across multiple SLO objects from the SLO wizard. The wizard creates individual SLOs; the weighted roll-up is something you compute yourself — as a DQL expression on a dashboard, or in a notebook like this one. Treat the number above as a reporting calculation you maintain, not a built-in SLO type. (This is a point the original field document glossed over.)

5. Rolling vs Calendar Windows

Window	Behaviour	Use when
Rolling (e.g. last 30 days)	Always moving; a bad day ages out gradually	Operational trend awareness, on-call signal — the default
Calendar (e.g. this month)	Resets at a boundary; budget refills on the 1st	Business reporting, contractual periods, exec reviews

Rolling windows give smoother, more honest day-to-day signals and avoid the "budget resets tomorrow so ignore today" trap. Calendar windows align to how the business reports. Many teams run both: rolling for alerting, calendar for the monthly review.

Whatever you choose, **alert on sustained breach, not on momentary dips** — a single bad interval inside a 30-day window is noise. SLO-04 covers how multiwindow burn-rate alerting encodes exactly that.

> **Sources:** Service-Level Objectives (DT docs), Site Reliability Engineering — Error Budgets (Google SRE Book). Burn-rate query re-executed against a live tenant 08/28/2026. **Derived:** the weighted-global calculation is a reporting pattern, not a built-in Dynatrace SLO type.
